Part 3 – System Testing Documentation

Project Context

This section presents the **system testing activities** carried out for the **Smart Library System**. System testing is performed to validate the behavior of the **entire system as a whole**, ensuring that all components work together correctly through the system's public interface.

Purpose of System Testing

The main goals of system testing in this project are to:

- Verify that the complete system satisfies the specified functional requirements
- Validate end-to-end user workflows and real usage scenarios
- Ensure correct interaction across all system layers
- Confirm that business rules are enforced consistently at system level

System Testing Approach

System testing for the Smart Library System was performed by:

- Testing the system **end-to-end through the LibraryFacade**, which represents the system's public entry point
- Using **real implementations** of all components (storages, policies, services, and notifications)
- Avoiding mocks and stubs to reflect real system behavior
- Simulating realistic user actions such as borrowing, returning, and capacity enforcement

This approach ensures that the system behaves correctly from the user's perspective.

Test Organization and Responsibilities

- System tests are organized around **complete user scenarios**
- Each test validates a specific system-level behavior or rule
- Team members were assigned 4 system test cases each, based on scenario complexity
- Each responsible member implemented their assigned tests and documented the results in the table below

System Testing Documentation Table

Class: LibrarySystemTest
Responsible Members: All Group

Test Case	Test Description & Reason	Expected Output	Actual Result
Arsildo Veliu			
borrowFlow_success_marksItemUnavailable_andCreatesActiveBorrowRecord()	Verifies that a successful borrow operation marks the item as unavailable and creates an active borrow record in the system	Item marked unavailable, active borrow record created	Item unavailable, borrow record created
returnFlow_success_marksRecordReturned_andMarksItemAvailable()	Verifies that a successful return operation marks the borrow record as returned and restores item availability	Item marked available, borrow record marked returned	Item available, record returned
borrowFlow_nonexistentMember_throwsRuntimeExceptionWithMessage()	Ensures the system prevents borrowing when member does not exist and provides clear error message	RuntimeException with "Member not found" message	Exception thrown with correct message
borrowFlow_nonexistentItem_throwsRuntimeExceptionWithMessage()	Ensures the system prevents borrowing when item does not exist and provides clear error message	RuntimeException with "Item not found" message	Exception thrown with correct message
Megi Almadhi			
completeUserJourney_borrowMultipleItems_returnSome_borrowMore()	Tests a realistic user workflow: borrow multiple items, return some, borrow additional items. Verifies capacity tracking across multiple operations	Active borrows correctly tracked through lifecycle: 0→2→1→2	Borrow count changes correctly through all operations

studentMember_canBorrow3Items_cannotExceed()	Verifies that STUDENT members can borrow up to their limit (3) and are prevented from exceeding it	3 items borrowed successfully, 4th item rejected with policy violation	Limit enforced correctly
borrowUpToLimit_returnOne_borrowAnother_respectsCapacity()	Tests capacity management: GUEST borrows to limit (1), cannot borrow more, returns item, then can borrow different item	Borrow blocked at capacity, allowed after return	Capacity respected throughout workflow
borrowReturn_sameItem_multipleTimes()	Tests that the same item can be borrowed and returned multiple times, creating separate borrow records	3 separate borrow records created, all marked returned	Multiple borrow-return cycles work correctly
Sidrit Rrustemi			
staffMember_canBorrow5Items_cannotExceed()	Verifies that STAFF members can borrow up to their limit (5) and are prevented from exceeding it	5 items borrowed successfully, 6th item rejected with policy violation	Limit enforced correctly
guestMember_limitedTo1Item()	Verifies that GUEST members can borrow up to their limit (1) and are prevented from exceeding it	1 item borrowed successfully, 2nd item rejected with policy violation	Limit enforced correctly
twoMembers_cannotBorrowSameItem_secondMemberBlocked()	Tests item contention: when one member borrows an item, another member (even with higher privileges) cannot borrow it until returned	Second borrow blocked, succeeds after return	Item availability prevents concurrent borrows
borrowFlow_inactiveMember_throwsRuntimeException()	Ensures inactive members are prevented from borrowing items	RuntimeException with "policy violation" message	Inactive status blocks borrowing

Viola Zeneli			
twoMembers_borrowAndReturn_differentItems_noInterference()	Verifies that multiple members can borrow different items simultaneously without interfering with each other's operations	Member 1 returns item, Member 2's borrow unaffected	Members operate independently
memberWithOverdueItem_cannotBorrowAnother()	Tests that members with overdue items are blocked from borrowing additional items until overdue items are returned. NOTE: Creates overdue state manually due to time dependency limitation	Borrow blocked with "policy violation" when overdue exists	Overdue status prevents new borrows
returnOverdueItem_sendsAlert()	Verifies that returning an overdue item triggers an alert to the member with fine information. NOTE: Creates overdue state manually due to time dependency limitation	Alert sent to member with overdue message and fine	Alert service called with correct member ID and message
systemMaintainsConsistency_itemAvailabilityMatchesBorrowRecords()	Verifies data consistency across the system: item availability status matches borrow records state	Item availability consistent with active borrow records	System maintains consistency between storage layers

